



CS683: Advanced Computer Architecture-2024

L8-The final frontier

<https://www.cse.iitb.ac.in/~biswa/courses.html>

<https://www.cse.iitb.ac.in/~biswa/>



Phones (smart/non-smart)
on silence plz, Thanks

Pipelined Caches (mostly for L1 caches)

- Comparing index bits, comparing multiple tags, and then respond with the data
- Can we pipeline it? 3-stage pipeline will do
- This will amortize access latency 😊
- Banked cache provides bandwidth 😊

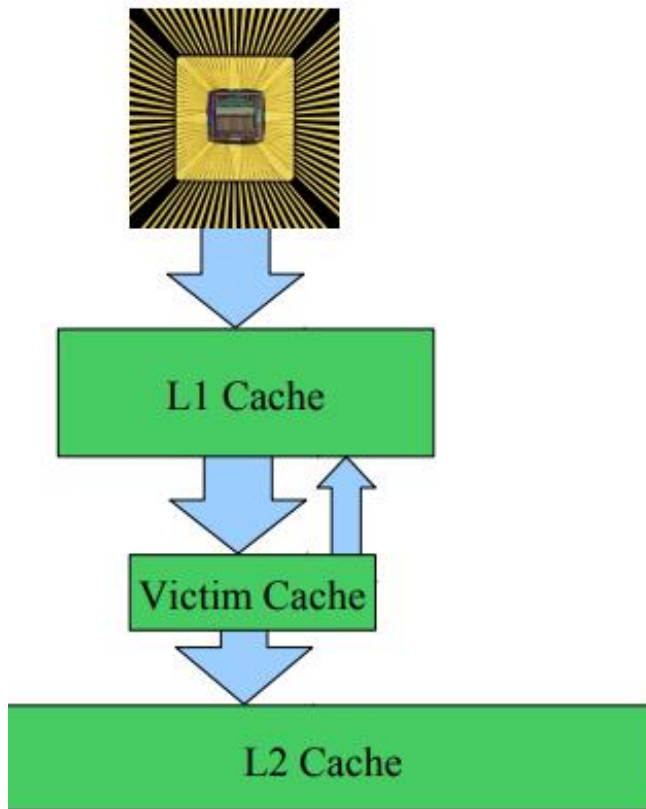
Stream buffer [ISCA 1990]

- Put a buffer between two levels of cache say L1 and L2.
- On a miss to address A1 at L1, prefetch A2 AK into a buffer called stream buffer. Future accesses for A2 to AK will be hit in the buffer.
- What kinds of misses eliminated? Compulsory, capacity 😊

How different from prefetch into cache?

- ISCA 1990 paper does not prefetch into the cache, instead prefetch it into the buffer.
- It acts like a next line prefetcher with degree k and fill the stream buffer.
- On a miss in the cache and the stream buffer, refill the stream buffer.
- Highly effective for many access patterns.

Victim Cache [To eliminate conflict misses]



Put a small fully-associative cache between two levels of cache, say L1 and L2.

On eviction, put the data of evicted addresses into this victim cache

Helps in improving conflict misses of associative L1 caches.

Poor man's associativity

- Not all sets suffer conflicts; provide limited capacity for conflicts

The paper to read if curious

- **Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers, ISCA 1990**

What Else?

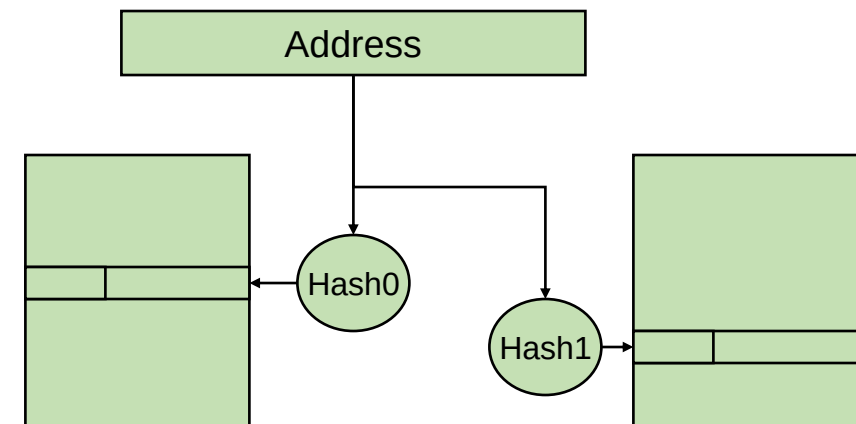
- How can you reduce conflict misses, further without changing associativity, size, or block size?

Seznec (my guru)'s skewed associative cache [ISCA 1993]



- Have different mappings for the two (or more) banks of the set-associative cache
- First mapping conventional; second one: dispersing: the addresses (XOR a few bits)
- Hit ratio of 2-way skewed as good as 4-way conventional.

<https://dl.acm.org/doi/pdf/10.1145/165123.165152>



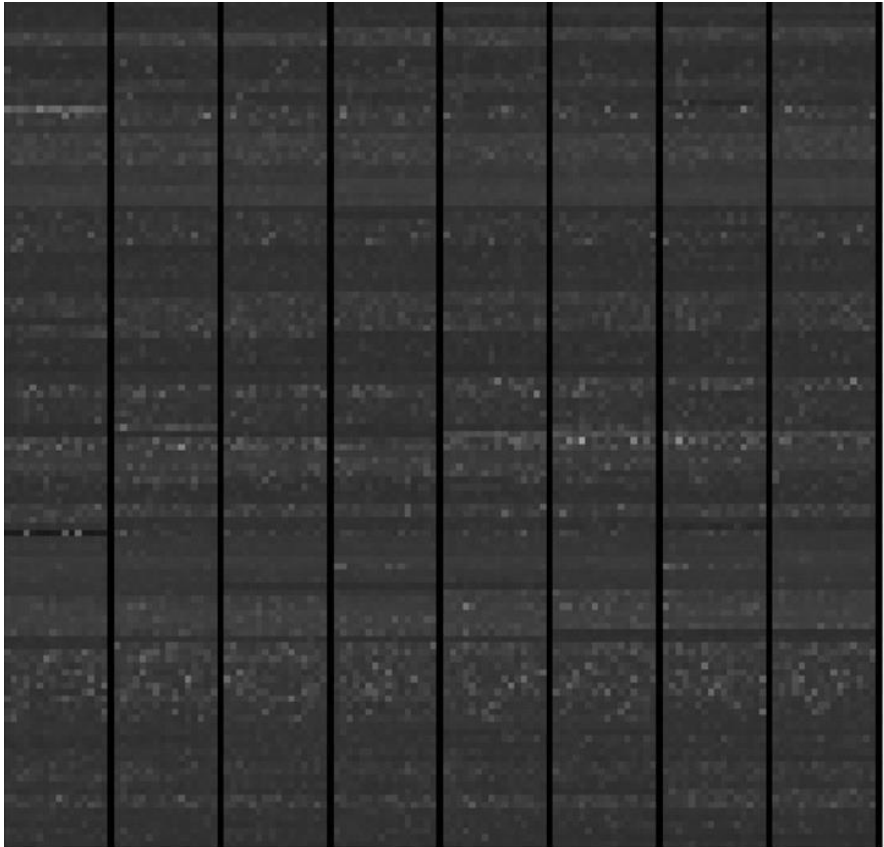
A detailed read: Skewed caches

- A blog from one of students, architecture course@IITK, 2019:
<https://medium.com/@arpitguptarag/two-way-skewed-associative-caches-9bab2c36dcee>

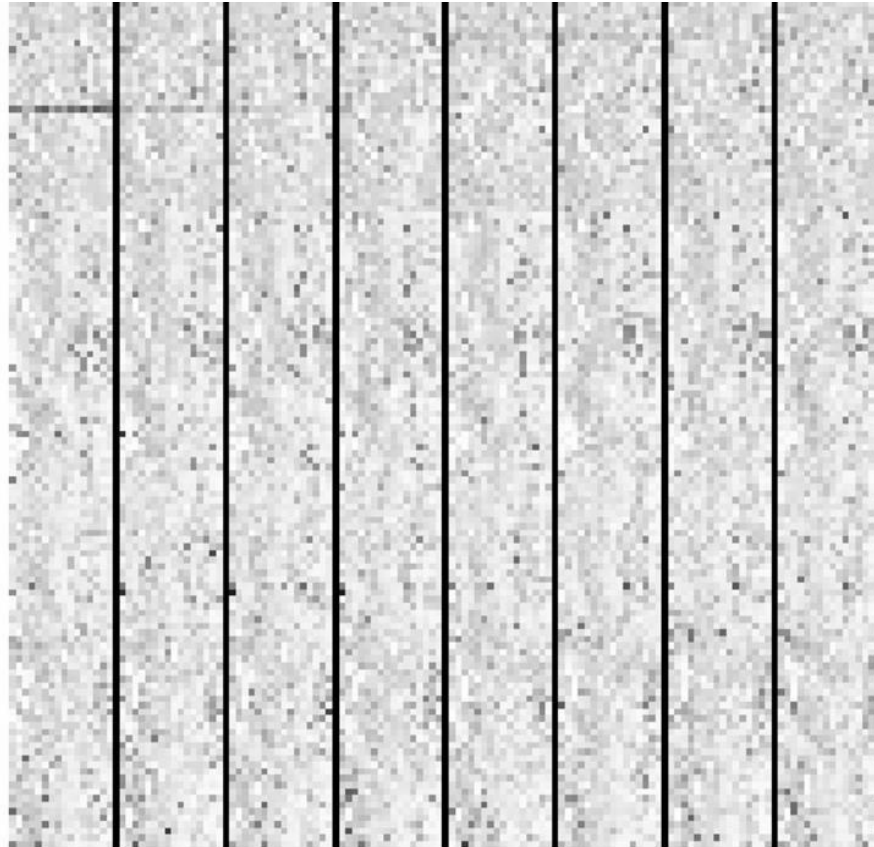
What else? Dead-blocks and Reuse Cache

- L3 caches are larger in size.
- Large fraction of lines are dead on arrival.
- Why cache it if they wont get a reuse? Just bypass the L3 cache, and directly put it in L2 cache.
- But how to predict, which one will be dead or which one will be reused?
- Dead block predictors, Reuse Cache

Dead Block Predictors



(a)



(b)

Greyscale		Efficiency
		0.0
		0.1
		0.2
		0.3
		0.4
		0.5
		0.6
		0.7
		0.8
		0.9
		1.0

Each pixel = average live time of a cache block, goal: to get b from a

More details on the predictor

- Ideally, the predictor should predict for each cache line address, whether that address will be reused or not.
- How? By looking at the past, put some saturating counters to gain confidence on the prediction for each address.
- However, this is not practical and will lead to significant storage overhead.

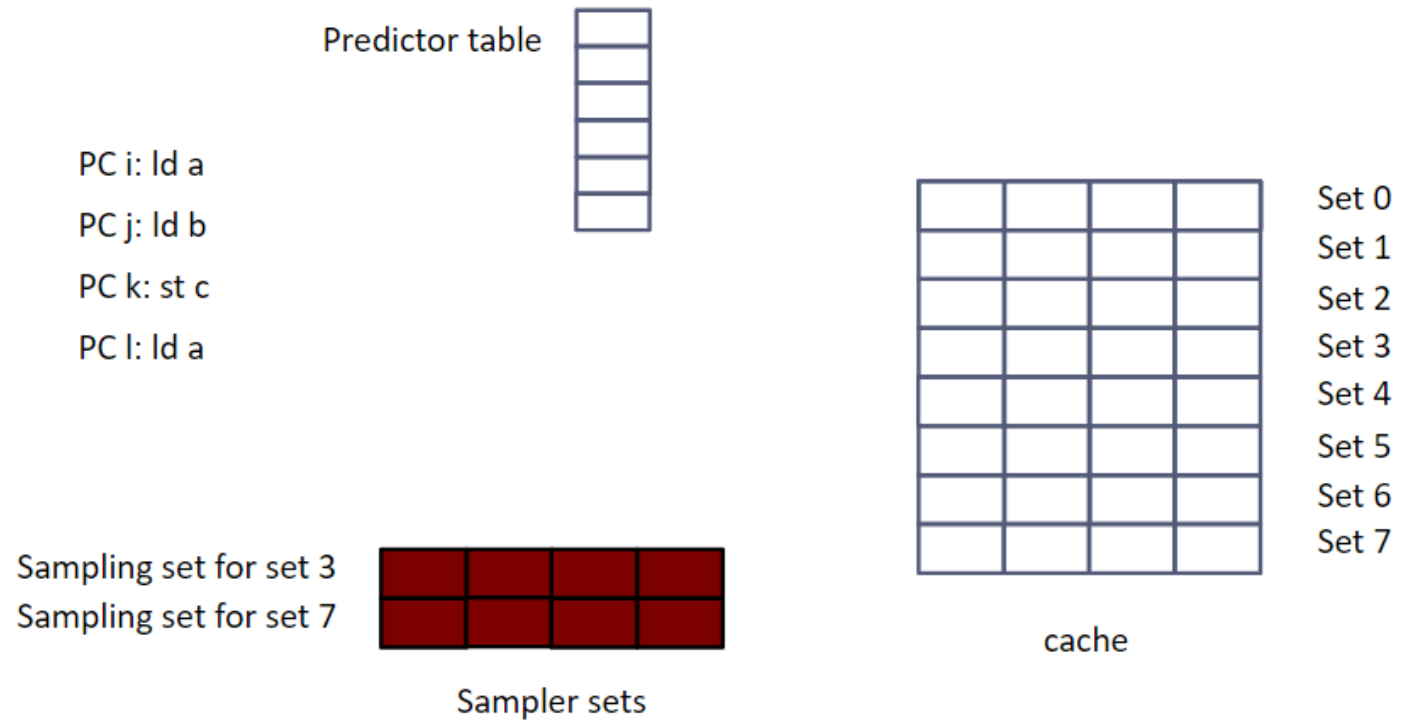
The practical dead block predictor

- 80 to 90% of data accesses are come from 10% of instructions (program counters).
- Instead of predicting the usefulness of each block, we can predict, the usefulness of all the cache line addresses brought by a single PC.
- The predictor will now be indexed by PC and saturating counters will be used for per PC entry.

Devil is in the details

- Please note that with this implementation, we need the PC information per cache line as we need to check whether the line was reused at the time of eviction, and we need the PC of the evicted cache line to index into the predictor.
- However, storing PCs along with the cache lines will lead to storage overhead again. Instead, we can use few sets (sampler sets) where we will store PCs with the cache lines and use that for prediction. For rest of the sets (follower sets), no need to store PCs and no need to learn their reuse.

The complete story



Only 32 sampler sets in 2MB LLC

Let's bring the real Industry numbers

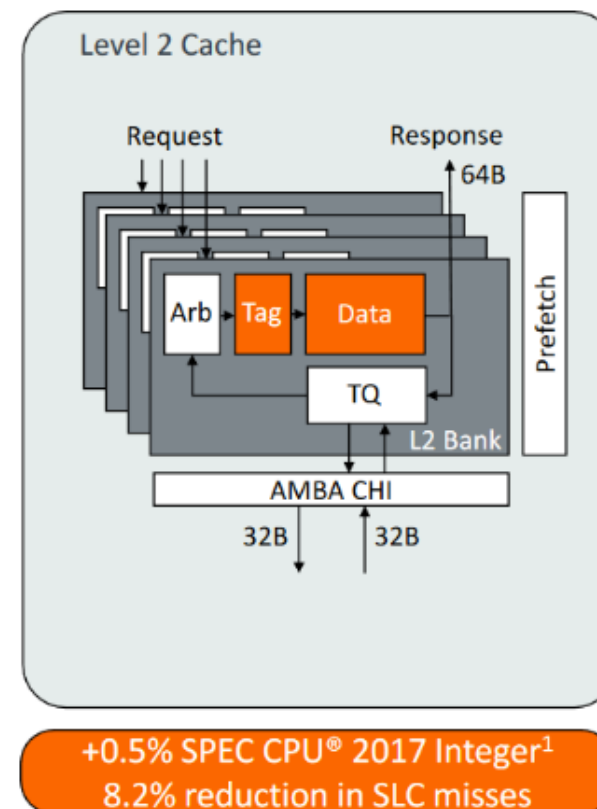
Level 2 Cache

uArch features shared with Neoverse V1

- + Private unified Level 2 cache, 8-way SA, 4 independent banks
- + 64B read or write per 2 cycles per bank = 128B/cycle total
- + 96-entry Transaction Queue
- + Inclusive with L1 caches for efficient data and instruction coherency
- + Inline SECDED ECC in Tag, Data, and TQ RAMs
- + AMBA CHI interface with 256b DAT channels

uArch features **new with Neoverse V2**

Capacity	2MB/8-way with latency of 1MB (10-cycle Id-to-use)
Replacement policy	6-state RRIP (up from 4)
Dead-block prediction	Separate tracking of used-once and used-multiple data
Replacement	L2 copybacks transfer replacement hints to SLC
CHI revision E interconnect	Improved store-hit-shared flow (MakeReadUnique) Combined Write/Cache Maintenance transactions Write*Zero transactions for memset



What about prefetchers?

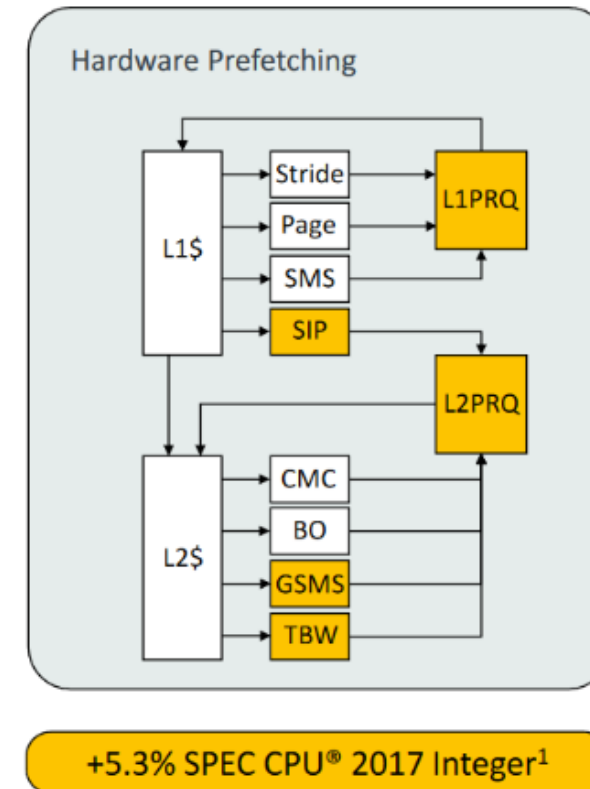
Hardware Prefetching

uArch features shared with Neoverse V1

- + Multiple prefetching engines training on L1 and L2 accesses
 - + Spatial Memory Streaming
 - + Best Offset
 - + Stride
 - + Correlated Miss Cache
 - + Page
- + Prefetch into L1 and L2
- + Virtual address to allow page crossing and TLB prefetching
- + Adaptive distance based on accuracy and timeliness

uArch features **new with Neoverse V2**

Training	Refined filtering of transactions used for training
Accuracy	Apply Program Counter to L2 GSMS training
New PF engines	Global SMS – larger offsets than SMS Sampling Indirect Prefetch – pointer dereference TableWalk – Page Table Entries
Differentiated QoS	Lower QoS value for prefetches than demand for reduced loaded latency





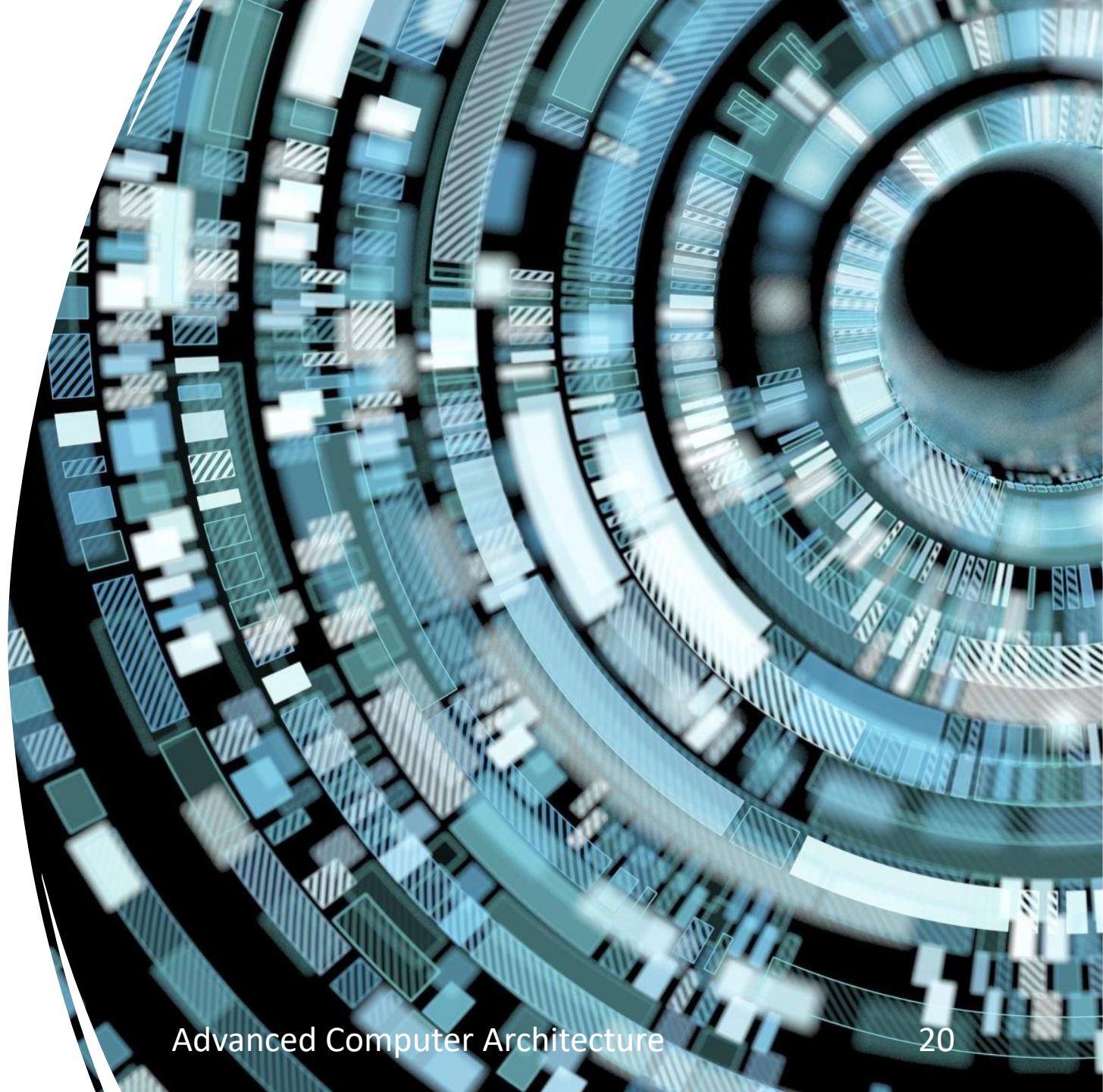
The Maya of Virtual world

<https://www.cse.iitb.ac.in/~biswa/>

OS

- What is it?
- OS: The world of Maya

Virtual world: Virtual memory,
Process (virtual CPUs)



Operating System and Architecture: Bandish 101

Case 1: Programmer wants to run 100 things

CPU says I am alone 😞

OS says I can create an illusion of multiple CPUs 😊

Case 2: Programmer wants 100s of GBs of data

Memory says I am just 10 GB 😞

OS says, never mind, I can create an illusion of TBs 😊

Operating System and Architecture: Bandish 101

Case 3: Programmer wants protection/security of data

OS says I can do it but need your support 😞

CPU says sure 😊

Case 4: Programmer wants parallelism

OS says, why not 😊 but the cost of parallelism 😞

CPU says I will take care 😊
